

Siege 1618: design document → database

A game design document, informal and ambiguous in exactly the places that matter — which is what a real specification looks like. Take it through the pipeline: ER model, relational schema, DDL, and the queries the team will actually run.

Names _____ Section _____ Date _____

SECTION 1

The mini-world

Siege 1618 is a two-player, two-screen siege game. Each player holds a terrain map, raises a castle, harvests resources, trains units, and attacks the other side.

Stated in the document

- **Resources** — wood, stone, crops, gold; a tile position and an amount remaining. Serfs harvest them.
- **Units** — serf, archer, cannon; position, hit points, attack, speed in tiles/sec, range.
- **Structures** — castle, tower, wall; same, plus rate of fire.
- **Terrain** — tiles, some holding resources; a level file.
- **Production** — a structure can produce a serf or an archer.

Not stated — you must decide

- **Players** and **matches** are never mentioned. Nothing says who owns a unit or which game it is in.
- Attack and speed are listed as properties of a *unit*. Are they?
- An attack's attacker and target can each be a unit *or* a structure.
- Nothing records history, only current state. So: "how much damage did she deal last match?"

SECTION 2

How the session runs

Work in pairs — one drives the diagram, the other argues with it; swap at Part 3. Paper until Part 4, then your MySQL instance.

Part	What you produce	Time
1	ER diagram — entities, keys, relationships, cardinality	20 min
2	Relational schema from your diagram	15 min
3	<code>CREATE TABLE</code> for three relations	10 min
4	SQL for the question set	25 min

Part	What you produce	Time
—	Debrief: whose model made Part 4 easier?	5 min

Two things before you start

Credit is for the **model**, not for having every query run — a model that makes the queries impossible is what we are testing for.

The last page carries a reference schema. **Do not turn to it until Part 2 is finished** — the point is the gap between what you build and what you needed to have built.

SECTION 3

Three decisions the document does not make

3.1 — Base values and current values

An archer *type* moves at 1.2 tiles/sec — the designer's number, in a balance file. *This* archer, shot down to a third of its hit points, does not. A damaged cannon fires weaker and shorter. So every combat stat exists twice: a **base value** on the type and a **current value** on the instance. The document half-admits it for one of them — "hit points: *current* hit points" — and the mistake is to treat HP as the special case instead of the example.

On the type — edited in a balance file	On the instance — drifts with every hit
BaseMaxHP, BaseAttack, BaseSpeed, BaseRange, BaseRateOfFire, build cost	CurrentHP, CurrentAttack, CurrentSpeed, CurrentRange, CurrentRateOfFire, position, owner, match, destroyed-at

The follow-on question

Must you *store* every current value? If `CurrentSpeed` is a pure function of `CurrentHP` and `BaseSpeed`, it is a **derived attribute** — the dashed oval from Ch. 3 — computed on read. The moment an upgrade, terrain penalty or status effect can also move it, it stops being a pure function and must be stored. Decide which of yours are derived and which are stored.

3.2 — Does a tile exist without its level?

Tile (12, 7) is meaningless alone — there is a (12, 7) on every level and they are different places. That is a **weak entity**: no key of its own, identified by a partial key (`( X, Y )`) plus an owner's key, through an identifying relationship with total participation. Double rectangle, double diamond.

The harder half. A resource node sits at a tile; a unit stands at a tile. Relationships to TERRAIN_TILE, or plain `X / Y` attributes? It depends on whether anything queries by tile — **Q2** settles it.

3.3 — Who attacked whom?

An archer shoots a wall; a tower shoots a serf. Attacker and target may each be a unit or a structure, and ER notation has no "unit or structure." Every telemetry question runs through this relationship. Pick one and carry it through.

Design	What the DBMS enforces	Cost of "damage per player"
A · nullable FK pairs	One ATTACK relation, four optional FKs. "Exactly one of each pair is non-null" needs a <code>CHECK</code> ; much integrity falls to application code.	Two outer joins plus a <code>COALESCE</code> to recover one owner.
B · four relations	Every FK <code>NOT NULL</code> and fully enforced, but relations grow as the product of attacker kinds and target kinds.	A <code>UNION ALL</code> over all four before you can group.
C · GAME_OBJECT supertype	UNIT and STRUCTURE share one ObjectID space; ATTACK needs two <code>NOT NULL</code> FKs, both enforced. The supertype owns what is common.	Two inner joins, no nulls, no union. One extra join to reach a subtype attribute or a base value.

PART 1

ER model

20 min

TASK 1.1

Entities, attributes, keys

Apply the base/current split from §3.1 across the whole document. Expect nine to thirteen entity types — four means you have collapsed something, twenty-five means you have promoted an attribute.

1. For each entity type, list its attributes and **underline the primary key**. Mark composite, multivalued and derived attributes in Chen notation.
2. **Base and current.** For every combat stat — hit points, attack, speed, range, rate of fire — say which side it lands on and whether the current value is *stored* or *derived*. Draw derived ones as dashed ovals. Be ready to say what would force a derived one to become stored.
3. Two entity types have no natural key in the document. Say what you use instead and why.
4. **Position is a trap.** `X` and `Y` appear on resources, units and structures. Composite attribute, or is a tile an entity in its own right? Write one sentence justifying your choice — you will be held to it.

TASK 1.2

Relationships, cardinality, participation

Every relationship gets a cardinality ratio *and* a participation constraint. Say both out loud in each direction before writing them down.

1. A **player** and a **match**. What ratio? What must the relationship carry that neither entity can hold?
2. A **serf** and a **resource node**. A serf works the same node repeatedly; a node is worked by many serfs. What does that force?
3. A **structure** and the **units it produces**.
4. A **level** and its **terrain tiles**.
5. A **match** and a **level**. Which direction is total?

Watch for: two of these have an attribute belonging to the *relationship* and not to either entity. If you cannot find them, you have hidden them inside an entity — which will cost you in Part 4.

ENTITY TYPES — NAME EACH ONE AND ITS KEY BEFORE YOU DRAW

ER DIAGRAM — TASK 1.2

TASK 2.1

Run the mapping algorithm on your own diagram

Use the seven-step algorithm from Ch. 9. Write each relation as `NAME(key, attr, FK → TARGET)`.

1. Map regular entity types, then your weak entity — the owner's key joins the partial key to form the primary key.
2. Map relationships. For each 1:N, state which side absorbed the foreign key and why. For each M:N, name the cross-reference relation and what relationship attributes it carries.
3. Handle the multivalued attribute. (If you think you have none, look again at a player's stockpile across four resource types.)
4. If you chose design C in §3.3, map the specialization. Which of the four EER mapping options from Ch. 4 did you use, and is it total or partial, disjoint or overlapping?

Before you move on: count your relations and your foreign keys. Every foreign key should name the relation it points at. A column you cannot trace to a primary key is a bug, not a shortcut.

RELATIONAL SCHEMA

Stop here

Only now turn to the reference schema on the last page and compare. Where you differ, decide whether you were wrong or merely different.

TASK 3.1

Three **CREATE TABLE** statements

Write DDL for **TERRAIN_TILE**, **PLAYS_IN** and **ATTACK**. Each forces a different decision: a composite primary key that includes a foreign key; a cross-reference relation with its own attributes and a domain constraint; and two foreign keys into the *same* relation, which means you must name the constraints.

Checklist for each table

- Sensible types and widths; **NOT NULL** wherever participation is total.
- Primary key, plus any **UNIQUE** the cardinality demands.
- Foreign keys with a deliberate **ON DELETE** action.
- A **CHECK** where a domain is genuinely restricted.

Think about

Can a **CHECK** stop an archer attacking a target in a different match? Try to write it. What does the attempt tell you about where that rule has to live instead?

Part 3 · write your DDL here

TERRAIN_TILE

PLAYS_IN

ATTACK

PART 4

SQL 25 min

These are the questions the team actually asks — balance, telemetry, leaderboard. Before you type each one, say which relations it touches. **If a query needs a relation your Part 2 schema does not have, write that down** — it is the most valuable thing you will produce today.

Tier 1 — reading the catalog 5 min

- Q1** Every unit type with a base range of 4 or more, showing name, base attack and base speed, cheapest first by gold cost.
Single relation · WHERE · ORDER BY — you are reading the catalog, so every value here is a base value
- Q2** For the level named 'Highland Pass', how many tiles are there of each terrain type?
Join · GROUP BY — the query that settles the tile question in §3.2
- Q3** All gold nodes on that same level, richest first, with their tile coordinates.
Join · two WHERE conditions · ORDER BY DESC
- Q4** Which structure types can attack at all, and what is the ratio of their base attack to their base rate of fire?
Computed column · a thought about division by zero — a wall has no rate of fire

Tier 2 — match telemetry 11 min

- Q5** In match 7, how many units does each player still have alive?
Three-way join · GROUP BY · the NULL test that means "not dead"
- Q6** Every unit in match 7 now moving at less than 60% of its type's base speed, with the shortfall shown.
The base-versus-current join · a ratio in the WHERE clause — if your schema derives CurrentSpeed instead of storing it, write the formula here and notice what it costs you
- Q7** Total damage dealt by each player in match 7, highest first.
Join through the attacker · SUM · this is the query compared in §3.3
- Q8** For every unit type in the catalog, how many have ever been built across all matches — **including types never built at all**.
Outer join · the zero-row trap: an inner join silently drops the answer you most wanted
- Q9** The five resource nodes worked hardest across the whole database: number of harvest events and total amount taken.
GROUP BY · two aggregates · LIMIT
- Q10** Every player's record — matches played, won, and lost.
Conditional aggregation: SUM over a CASE, or COUNT of a filtered expression
- Q11** Which players hold more than 500 gold in any match currently in progress?
Join STOCKPILE · a NULL test on the match end · HAVING or WHERE — decide which and know why

Tier 3 — the designer's questions

9 min

- Q12** Damage per gold spent, by unit type, across all matches. Which type is the outlier the designers need to look at?
Aggregate divided by a base catalog value · ORDER BY an expression
-
- Q13** Win rate by starting side. Is one side of the map favored?
Conditional aggregation as a percentage — the question that ships a patch
-
- Q14** Units that dealt damage but never took any.
NOT EXISTS or a left-join-is-null anti-join · say which you expect to be faster and why
-
- Q15** Average time from the start of a match to that player's first completed castle.
A correlated MIN · timestamp arithmetic · an aggregate over an aggregate
-
- Q16** For each match, the structure that stood longest before it was destroyed.
Greatest-n-per-group: a window function or a correlated subquery · what do you do about structures still standing?
-
- Q17** Players whose average final score beats the average final score of all players.
Nested aggregation · the classic "aggregate of an aggregate" that cannot be written in one GROUP BY

GRADUATE SECTION

CS 510 addendum

TASK G.1

Relational algebra

Express **Q5**, **Q8** and **Q14** in relational algebra as well as SQL. For Q8, name the operator that does the work an outer join does, and say what the algebra gives you for the types with no units built.

TASK G.2

Justify the attack design

You have three candidate designs from §3.3 and a stated workload: Q7, Q14 and Q16 run on every match-end screen. Argue for one design against that workload. Name what each alternative costs you — in enforced integrity, in query complexity, and in what happens when a third kind of attacker is added next release.

TASK G.3

The schema that already exists

The rendering client was written before any of this and reads a flat level file: one row per tile, with resource type and amount folded into the tile row, and no notion of a match. You may not change it — it ships on consoles. Specify the **proxy layer** that lets the client keep reading its shape while the server writes to yours: which views, which materialized views, which bridge tables. Justify each choice against the workload above.

Note: both schemas stay intact. The proxy layer is the deliverable and the justification is what gets graded.

SECTION 5

Hand in

1. ER diagram, with every edge labeled for cardinality and participation.
2. Relational schema, keys underlined, foreign keys named with their targets.
3. DDL for TERRAIN_TILE, PLAYS_IN and ATTACK.
4. A `.sql` file with as many of Q1–Q17 as you reached, in order, each preceded by the question as a comment.
5. One paragraph: **the query you could not write against your own Part 2 schema**, and what you would change.

On item 5

It is worth as much as the rest. Finding the question your model cannot answer is the skill; writing the model that answers everything was never going to happen on the first try. The relationship between workload and schema is most of what database design is in practice.

ITEM 5 — THE QUERY MY SCHEMA COULD NOT ANSWER

Do not read this page until Part 2 is finished.

One defensible answer, taking design C from §3.3. Hold yours next to it: where you differ, decide whether you were wrong or merely different.

APPENDIX

Reference schema

```
PLAYER(PlayerID, Handle, JoinedOn)
LEVEL(LevelID, Name, Width, Height)
TERRAIN_TILE(LevelID → LEVEL, X, Y, TerrainType)
RESOURCE_NODE(NodeID, LevelID → LEVEL, X, Y, ResourceType, InitialAmount)
MATCH(MatchID, LevelID → LEVEL, StartedAt, EndedAt)
PLAYS_IN(MatchID → MATCH, PlayerID → PLAYER, Side, FinalScore, Outcome)
UNIT_TYPE(TypeCode, Name, BaseMaxHP, BaseAttack, BaseSpeed, BaseRange,
          CostWood, CostStone, CostCrops, CostGold)
STRUCTURE_TYPE(TypeCode, Name, BaseMaxHP, BaseAttack, BaseRateOfFire,
               BaseRange, CostWood, CostStone, CostCrops, CostGold)
GAME_OBJECT(ObjectID, MatchID → MATCH, OwnerID → PLAYER, X, Y,
            CurrentHP, CurrentAttack, CurrentRange, CreatedAt, DestroyedAt)
UNIT(ObjectID → GAME_OBJECT, TypeCode → UNIT_TYPE, CurrentSpeed)
STRUCTURE(ObjectID → GAME_OBJECT, TypeCode → STRUCTURE_TYPE, CurrentRateOfFire)
STOCKPILE(MatchID, PlayerID, ResourceType, Amount)  FK (MatchID, PlayerID) → PLAYS_IN
HARVEST(HarvestID, ObjectID → UNIT, NodeID → RESOURCE_NODE, Amount, HarvestedAt)
ATTACK(AttackID, AttackerID → GAME_OBJECT, TargetID → GAME_OBJECT, Damage, OccurredAt)
PRODUCTION_ORDER(OrderID, StructureID → STRUCTURE, TypeCode → UNIT_TYPE,
                  OrderedAt, CompletedAt)
```

Four things to notice

Every **Base*** name has a **Current*** counterpart on the instance side, and the shared ones sit on the supertype — only **CurrentSpeed** and **CurrentRateOfFire** are subtype-specific. This version *stores* all current values; if yours *derives* some, say from what.

DestroyedAt and **CompletedAt** are null while the thing is alive or pending — that is how current state and history live in one relation.

STOCKPILE is the multivalued attribute, mapped out.

PLAYS_IN carries **Side**, **FinalScore** and **Outcome**, none of which belong to a player or to a match alone.